

Comprendre les règles d'accès et la sécurité des applications

Objectif :

Maîtriser les concepts de contrôle d'accès, d'authentification, de protocoles sécurisés et de cryptage afin de renforcer la sécurité des applications.

1. Droits d'accès des utilisateurs

a. Rôles des utilisateurs

Définition : Un rôle regroupe un ensemble de permissions. Un utilisateur se voit attribuer un ou plusieurs rôles définissant ce qu'il est autorisé à faire dans l'application.

Exemples de rôles :

Rôle	Description
Admin	Gère tous les utilisateurs, données et paramètres système.
User	Gère uniquement ses propres données (lecture/écriture).
Guest	Accès limité à certaines ressources, uniquement en lecture.

b. Contrôle d'accès basé sur les rôles (RBAC)

Définition : Le *Role-Based Access Control* est une méthode de gestion des autorisations fondée sur les rôles attribués aux utilisateurs.

Principe :

- L'utilisateur reçoit un rôle (ex. : admin).
- Le rôle contient des permissions (ex. : supprimer un compte).
- L'application vérifie le rôle de l'utilisateur avant de lui accorder l'accès à une action.

Avantages :

- Sécurité renforcée.
- Facilité de maintenance.
- Réduction du risque d'erreurs humaines.

Exemple simple :

Action	Admin	User	Guest
Lire des données	v	v	v
Modifier ses données	v	v	x
Supprimer un utilisateur	v	x	x

c. Cas pratiques simples

1. Créer une table représentant les rôles et permissions dans une application.
2. Simuler une tentative d'accès : l'utilisateur "user" essaie de supprimer un compte. L'application refuse.
3. Écrire des règles d'accès en pseudo-code :

Si utilisateur.rôle == "admin" ALORS autoriser suppression

SINON refuser

2. Accès sécurisé aux données de l'application

a. Protocoles sécurisés

1. HTTPS (Hypertext Transfer Protocol Secure)

- Chiffre les données échangées entre client et serveur.
- Utilise TLS (anciennement SSL).
- Exemple : site e-commerce → <https://moncommerce.com>

2. SSH (Secure Shell)

- Utilisé pour se connecter à distance à un serveur en ligne de commande.
- Connexion sécurisée, souvent pour administrateurs systèmes.
- Exemple : `ssh admin@monserveur.com`

b. Authentification forte (multi-facteur)

Définition : Combinaison de plusieurs méthodes pour prouver l'identité d'un utilisateur.

3 facteurs classiques :

Type	Exemple
Ce que je connais	Mot de passe

Type	Exemple
Ce que je possède	Téléphone, clé USB de sécurité (token)
Ce que je suis	Empreinte, visage, rétine

Exemple concret : Connexion à un compte Google avec mot de passe + code SMS.

c. Stockage sécurisé des données sensibles

1. Mot de passe :

- Ne jamais stocker un mot de passe en clair.
- Toujours utiliser un algorithme de hachage sécurisé :
 - bcrypt
 - argon2
 - scrypt

2. Données sensibles (ex. : carte bancaire) :

- Utiliser un algorithme de chiffrement comme **AES-256**.
- Ne jamais stocker ces données si ce n'est pas nécessaire.

3. Fichiers de configuration :

- Ne pas exposer les clés d'API ou mots de passe dans le code.
- Restreindre les accès avec des permissions (lecture seule).

d. Cas pratiques simples

1. Simuler un login avec deux facteurs : mot de passe + code.
2. Visualiser un fichier JSON contenant des rôles et des droits.
3. Identifier les faiblesses d'un fichier de configuration contenant un mot de passe en clair.

3. Introduction au cryptage

a. Notions de base de cryptographie

1. Chiffrement symétrique :

- Une seule clé pour chiffrer/déchiffrer.
- Rapide mais nécessite un échange sécurisé de la clé.

- Exemple : **AES**

2. Chiffrement asymétrique :

- Clé publique pour chiffrer.
- Clé privée pour déchiffrer.
- Plus lent mais plus sûr pour l'échange.
- Exemple : **RSA**

b. Pourquoi chiffrer les données ?

- Empêcher l'interception ou le vol de données.
- Garantir la confidentialité des informations personnelles.
- Se conformer à la loi (RGPD, CNDP...).

c. Exemple concret

Une application stocke les numéros de cartes bancaires. En cas d'attaque, le pirate ne pourra rien exploiter si les numéros sont chiffrés avec une clé forte.